

CSC2044 Concurrent Programming in Java

Group Assignment

Concurrent Bank Account Transaction System

Item	Details
Course Code	CSC2044
Course Title	Concurrent Programming in Java
Assessment Type	Group Assignment
Assessment Weightage	25%
CLO Alignment	CLO4
Group Size	3 to 4 students
Submission Format	Zipped folder containing source code, report, screenshots, README file, and group contribution table

1. Course Learning Outcome Alignment

This assignment is aligned with CLO4, which focuses on the development of concurrent programming applications using multithreading and synchronization mechanisms.

In this assignment, students are required to design and implement a Java application that processes bank transactions using sequential and concurrent execution. Students must also demonstrate a race condition and apply synchronization to protect shared data. Therefore, the main focus of this assignment is practical implementation of Java concurrency concepts.

2. Assignment Overview

In this assignment, each group will develop a simple Concurrent Bank Account Transaction System using Java.

The system simulates a bank account that receives different types of transactions, such as deposits, withdrawals, balance checks, and service charges. The transactions will first be processed sequentially. The system will then be modified to process transactions concurrently using Java threads.

The assignment is designed to help students understand how multiple threads can access shared data and why synchronization is required when shared data is updated by more than one thread.

3. Assignment Scenario

A bank account may be accessed by different users or services at the same time. For example, one user may deposit money, another user may withdraw money, while the system may also check the balance or apply a service charge.

If all transactions are processed one by one, the execution order is fixed and easy to understand. However, real systems often need to handle several transactions at overlapping times. This can improve responsiveness, but it can also create incorrect results if multiple threads update the same account balance without proper control.

Your group is required to build a Java console-based application that demonstrates this situation and shows how synchronization can be used to protect shared data.

4. Assignment Objectives

After completing this assignment, students should be able to:

- Develop a Java application that processes transactions sequentially and concurrently.
- Use Java threads through Thread or Runnable.
- Demonstrate how multiple threads can access shared data.
- Identify a race condition in a shared bank account balance.
- Apply synchronized method or synchronized block to protect a critical section.
- Explain how synchronization improves correctness in a concurrent Java program.

5. Assignment Tasks

5.1 Task 1: System Design

Your group must briefly describe the bank account transaction system that you are going to build.

Your system should include at least four transaction types:

- I. Deposit transaction
- II. Withdrawal transaction
- III. Balance checking transaction
- IV. Service charge transaction

Your report should include a simple design table or diagram showing the main components of the system.

Component	Description
Bank Account	Stores the shared account balance
Transaction	Represents a deposit, withdrawal, balance check, or service charge
Transaction Worker	A thread or runnable task that processes a transaction
Shared Balance	The shared data accessed and updated by multiple threads

5.2 Task 2: Sequential Transaction Processing

Create a Java program that processes bank transactions sequentially.

Your program should:

- Create one bank account with an initial balance, for example RM1000.
- Create at least eight transactions.
- Process one transaction at a time.
- Print when each transaction starts.
- Print when each transaction finishes.

- Print the current balance after each transaction.
- Use `Thread.sleep()` to simulate transaction processing time.
- Show clearly that the next transaction starts only after the previous transaction finishes.

Example output style:

```
Starting Transaction 1: Deposit RM200
Finished Transaction 1
Current Balance: RM1200

Starting Transaction 2: Withdraw RM100
Finished Transaction 2
Current Balance: RM1100
```

In your report, briefly explain why this version is called sequential processing.

5.3 Task 3: Concurrent Transaction Processing Using Threads

Create another version of the program that processes transactions concurrently.

Your program should:

- Use either `Thread` or `Runnable`.
- Create at least four threads.
- Allow different threads to perform transactions on the same bank account.
- Print the thread name while processing each transaction.
- Use `Thread.sleep()` to simulate different processing times.
- Show that the output order may vary when the program is run multiple times.

Example output style:

```
Thread-1 started Deposit RM200
Thread-2 started Withdraw RM100
Thread-3 started Balance Check
Thread-4 started Service Charge RM50

Thread-3 finished Balance Check
Thread-1 finished Deposit RM200
Thread-4 finished Service Charge RM50
Thread-2 finished Withdraw RM100
```

In your report, compare the sequential and concurrent versions. Your explanation should include:

- Why concurrent output order may change.
- Why calling `start()` creates a new thread.
- Why calling `run()` directly does not create a new thread.

5.4 Task 4: Shared Data, Stack, and Heap Explanation

In your report, explain how your program uses private and shared data.

Your explanation should include:

- Which variables are local/private to a thread.
- Which objects or variables are shared by multiple threads.
- Why local variables are usually safer in multithreaded programs.
- Why the shared bank account balance can create problems.
- How the shared balance is stored as part of the bank account object.

5.5 Task 5: Race Condition Demonstration

Create an unsafe version of the bank account system to demonstrate a race condition.

Your unsafe version should:

- Use one shared bank account.
- Use at least two threads.
- Let each thread update the same balance many times.
- Use an update operation such as `balance = balance + amount;` or `balance = balance - amount;`
- Print the expected final balance.
- Print the actual final balance.
- Run the program several times and observe whether the result is always correct.

For example, if the initial balance is RM1000 and two threads each deposit RM1 for 100,000 times, then the expected final balance should be:

$RM1000 + RM100000 + RM100000 = RM201000$

However, without synchronization, the actual result may be less than the expected result.

In your report, explain:

- What the expected final balance should be.
- What actual value you received.
- Why the result may be incorrect.
- Why `balance = balance + amount` is not one single atomic operation.
- Why this problem is called a race condition.

5.6 Task 6: Synchronization Solution

Create a corrected version of the bank account system using synchronization.

Your corrected version should:

- Use a synchronized method or synchronized block.
- Protect the shared balance update.
- Print the final balance after all threads finish.
- Show that the final result is now reliable.

In your report, explain:

- Which part of your code is the critical section.
- Why only one thread should enter the critical section at a time.
- How synchronization prevents the race condition.

- Why synchronization is useful when multiple threads update shared heap data.

5.7 Task 7: Testing, Output Screenshots, and Reflection

Run your programs and include screenshots of the outputs.

Your screenshots should show:

- Sequential transaction processing output.
- Concurrent transaction processing output.
- Unsafe race condition output.
- Synchronized version output.

Your reflection should answer the following questions:

1. What is the main difference between sequential and concurrent transaction processing?
2. What did you observe when multiple threads performed transactions?
3. Why is the output order not always fixed in concurrent programs?
4. What problem happened in the unsafe bank account program?
5. How did synchronization solve the problem?
6. What did this assignment teach you about developing safe concurrent Java applications?

6. Required Submission

Each group must submit one zipped folder containing:

1. Java source code files.
2. A report in PDF format.
3. Screenshots of program outputs.
4. A README file explaining how to compile and run the program.
5. A group contribution table.

The zipped folder should be named using the following format:

CSC2044_Assignment_GroupXX.zip

Example: CSC2044_Assignment_Group05.zip

7. Suggested Report Structure

Your report should follow this structure:

1. Introduction
2. System Design
3. Sequential Transaction Processing
4. Concurrent Transaction Processing
5. Shared Data, Stack, and Heap Discussion
6. Race Condition Demonstration

7. Synchronization Solution
8. Testing, Screenshots, and Reflection
9. Group Contribution Table
10. References, if any

The report should be approximately 5–8 pages, excluding screenshots, appendices, and references.

8. README File Requirement

Your README file should briefly explain how to compile and run your program.

Example:

```
1. Open the project folder.

2. Compile the Java file:
   javac BankTransactionSystem.java

3. Run the program:
   java BankTransactionSystem
```

If your group uses multiple Java files, clearly explain the command needed to compile and run them.

9. Group Contribution Table

Each group must include a group contribution table in the report.

Student Name	Student ID	Main Responsibilities	Estimated Contribution (%)
Student 1			
Student 2			
Student 3			
Student 4, if applicable			

The total contribution percentage must add up to 100%.

All group members should understand the complete submitted work, not only the part they personally implemented.

10. Suggested Division of Work

Member	Suggested Responsibility
Member 1	System design and sequential transaction processing
Member 2	Concurrent transaction processing using threads
Member 3	Race condition and synchronization implementation
Member 4, if applicable	Report writing, screenshots, README, testing, and final checking

This division is only a suggestion. Groups may divide the work differently, but all members are expected to understand the complete system.

11. Marking Scheme

Component	CLO Alignment	Marks
Task 1: System design	CLO4	10
Task 2: Sequential transaction processing	CLO4	10
Task 3: Concurrent transaction processing using threads	CLO4	20
Task 4: Shared data, stack, and heap explanation	CLO4	10
Task 5: Race condition demonstration	CLO4	20
Task 6: Synchronization solution	CLO4	20
Task 7: Testing, screenshots, and reflection	CLO4	5
Report quality, README, and group contribution table	CLO4	5
Total		100

The total mark for this assignment will be converted to 25% of the course assessment.

12. Detailed Rubric

Criteria	Excellent	Good	Satisfactory	Weak
System design	Clear design with appropriate bank account, transaction, and worker components	Mostly clear design with minor gaps	Basic design only	Weak or unclear design
Sequential processing	Correctly shows one-by-one transaction processing with clear balance updates	Mostly correct with minor issues	Partially correct	Missing or incorrect
Concurrent processing	Correctly uses multiple threads and shows variable output order	Mostly correct thread implementation	Basic thread use only	Incorrect or unclear thread use
Shared data explanation	Clearly identifies private data, shared data, stack, and heap usage	Mostly correct explanation	Basic explanation only	Confused or missing explanation
Race condition demonstration	Clearly demonstrates unsafe shared balance update and explains incorrect result	Demonstrates race condition with minor limitations	Basic attempt but explanation is weak	Missing or incorrect
Synchronization solution	Correctly protects the critical section and shows reliable result	Mostly correct synchronization	Limited synchronization explanation	Missing or incorrect
Testing and	Strong screenshots and clear comparison	Good screenshots	Basic screenshots or	Weak or

reflection	of all versions	and explanation	reflection	incomplete
Submission quality	Complete, organised, easy to run, with clear README and contribution table	Mostly complete and organised	Some missing details	Poorly organised or incomplete

13. Important Notes

- The submitted work must be your group's own work. You may refer to lecture examples, but your final program and explanation should be written in your own way.
- The program does not need to have a graphical user interface. A console-based Java program is sufficient.
- Your focus should be on developing a working concurrent Java application and explaining how synchronization protects shared data.
- The unsafe race condition program may not produce an incorrect result every single time because thread scheduling is unpredictable. You should run it several times and include screenshots showing your observations.
- The synchronized version should produce a reliable final result because the shared balance update is protected.

14. Submission Deadline: 11:59 p.m. Malaysian Time, 19th June 2026